`sha256` to optimize storage and bandwidth, shown in figure 11.3. Before the application sends the query to the database, it takes the SQL query as the key to ask the caching layer for data—step 2. If the cache does not contain data for the key (step 3), the SQL query is sent to the relational database (step 4). The result (step 5) is then stored in the cache using the SQL query as the key (step 6). The next time the application wants to perform the query, it asks the caching layer (step 7), which now contains the cached table (step 8).

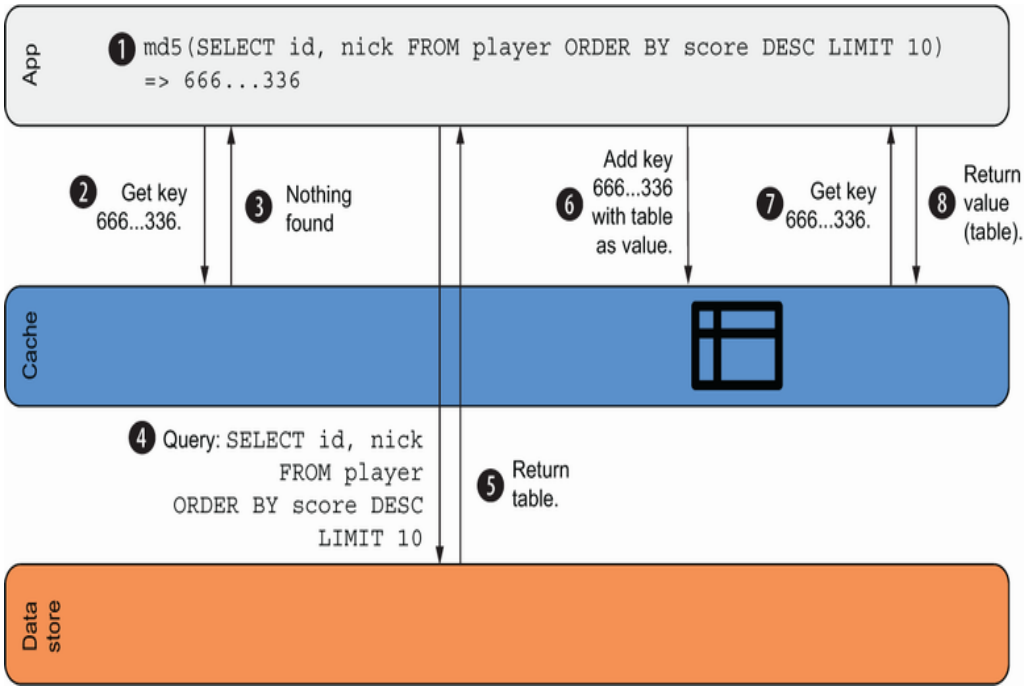


Figure 11.3 SQL caching layer implementation

To implement caching, you only need to know the key of the cached item. This can be an SQL query, a filename, a URL, or a user ID. You take the key and ask the cache for a result. If no result is found, you make a second call to the underlying data store, which knows the truth.

With Redis, you also have the option of storing the data in other data structures such as a sorted set. If the data is stored in a sorted set, retrieving the ranked data will be very efficient. You could simply store players and their scores and sort by the score. An equivalent SQL command would be as follows:

```
ZREVRANGE "player-scores" 0 9
```